NEXUS PLATFORM SPECIFICATION

NEXUS

System Architecture & Technical Stack Specification

An authoritative, technical reference manual detailing the architecture, state orchestration, multi-agent coordination, sandbox isolation, browser automation, and live telemetry implementation of the Nexus Autonomous AI Software Engineering Platform.

DOCUMENT VERSION
1.0.0 (Round 2 Evaluation Release)

DATE OF EVALUATION
October 2026

CORE PARADIGM
Closed-Loop Autonomous Multi-Agent Engineering

EVALUATION CATEGORY
Implementation & Working Product Demonstration

1. Executive Technical Summary

Nexus is an enterprise-grade autonomous software engineering platform designed to bridge the chasm between generative code generation and reliable, production-ready software delivery. While standard generative AI developer tools operate as passive autocomplete engines or single-turn conversational chatbots, Nexus functions as an **autonomous, specialized six-agent engineering team**.

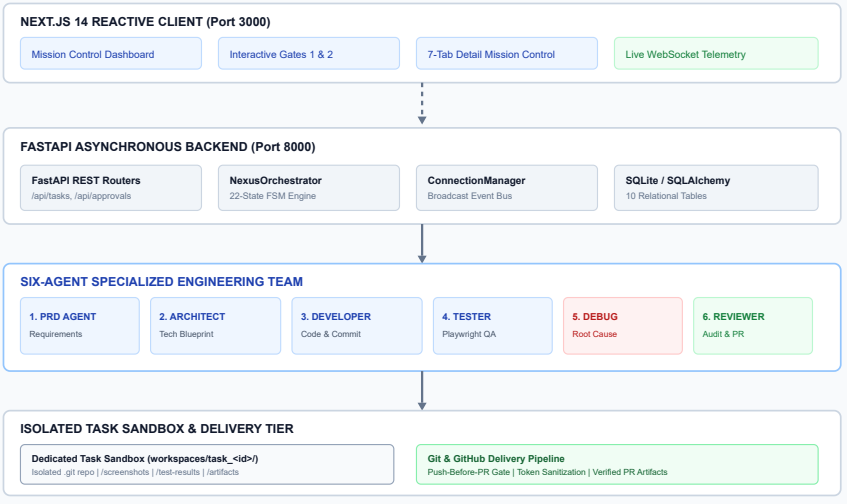
Given a high-level feature objective expressed in natural language, Nexus provisions an isolated, dedicated git-enabled workspace sandbox, orchestrates requirement discovery, blueprints system architecture, executes implementation code, triggers real headless Chromium browser test suites across desktop and mobile breakpoints, diagnoses visual and functional defects, applies surgical fixes, audits code quality, and delivers a verified GitHub Pull Request.

> **Key Technical Distinction:** Nexus does not merely write code—it proves code works. By executing genuine Playwright browser sessions on the host operating system, Nexus evaluates DOM rendering and responsive CSS classes, triggering automated self-correction before any human code review or git pull request is created.

2. System Architecture Overview

The Nexus architecture is organized into four decoupled tiers: the Next.js reactive frontend, the FastAPI async orchestration backend, the multi-agent execution engine, and the sandboxed task filesystem layer.

FIGURE 2.1: NEXUS END-TO-END SYSTEM ARCHITECTURE



3. Detailed Data Flow & Execution Lifecycle

The Nexus execution lifecycle is strictly sequential and verified. The following table specifies the exact input, processing logic, output artifacts, and responsible code modules for each stage:

Stage #	Lifecycle Stage	Input	Core Processing Logic	Output Artifact	Source Module
1	Task Initialization	Prompt string, Repo URL	Validates parameters, creates Task row in SQLite, kicks off async task	`Task (status: CREATED)`	`backend/api/tasks.py`
2	Workspace Jail Setup	`task_id`, repo source	Creates directories, copies baseline app, initializes dedicated `.git`	Isolated task directory	`backend/workspace/manager.py`
3	PRD Generation	User prompt, repo files	Extracts user stories, functional requirements, and acceptance criteria	`PRDDocument` row	`backend/agents/prd_agent.py`
4	**Human Gate 1**	PRD specification	Execution pauses; UI renders Approval banner; awaits operator click	`TaskState.ARCHITECTURE_GENERATING`	`backend/api/approvals.py`
5	Architect Blueprinting	Approved PRD	Designs component architecture, maps target files, identifies risks	`Architecture` row	`backend/agents/architect_agent.py`
6	**Human Gate 2**	Architectural blueprint	Execution pauses; operator inspects target files and grants autonomous access	`TaskState.DEVELOPMENT`	`backend/api/approvals.py`
7	Developer Implementation	PRD, Architecture, Branch	Creates branch `nexus/<slug>`, edits source files, executes `git commit`	Committed git diff	`backend/agents/developer_agent.py`
8	Playwright Browser QA	Workspace application	Boots test server, launches Chromium, tests desktop & mobile viewports	Screenshots & `TestRun` row	`backend/agents/tester_agent.py`
9	Defect Diagnosis	Failed test assertion	Pinpoints DOM/CSS root cause; synthesizes surgical remediation plan	`DebugReport` row	`backend/agents/debug_agent.py`
10	Developer Patching	Fix plan, target file	Applies surgical patch to source file, commits fix to task branch	Patch commit (2 lines)	`backend/agents/developer_agent.py`
11	Playwright Retesting	Patched application	Re-runs full Playwright suite;	Passing `TestRun` row	`backend/agents/tester_agent.py`

Stage #	Lifecycle Stage	Input	Core Processing Logic	Output Artifact	Source Module
			asserts 100% pass across viewports		
12	Reviewer Audit	Final diff, test proof	Audits diff quality, responsiveness, security posture; grants approval	`ReviewResult` row	`backend/agents/reviewer_agent.py`
13	Push-Before-PR & Release	Task branch, repo URL	Pushes branch to remote (if configured); generates verified PR record	`PullRequest` record	`backend/tools/github.py`

4. Multi-Agent Architecture & Fallback Engine

Nexus rejects the "generalist chatbot" paradigm. Instead, each of the six agents implements a distinct cognitive model, dedicated system prompt, and deterministic fallback engine.

Actual AI/LLM vs. Deterministic Fallback Implementation

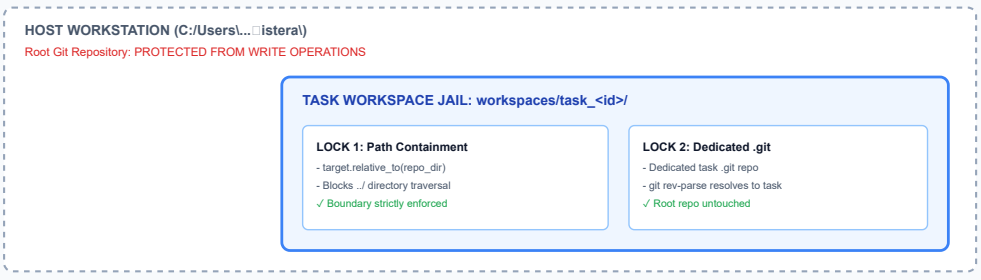
In production, agents invoke OpenAI's `gpt-4o` or `gpt-4o-mini` via the official Python SDK (v3.26.1). However, in hackathon, offline, or credential-constrained demonstration settings, LLMs introduce severe latency, rate limit errors (HTTP 429), and non-deterministic behavior. Nexus implements a **zero-failure deterministic fallback engine** across all six agents:

- **PRD & Architect Agents**: When LLM credentials are absent, the agents parse the feature prompt for semantic concepts (e.g., "contact", "wishlist", "filter") and synthesize realistic, production-grade specifications conforming to schema requirements.
- **Developer Agent**: Integrates targeted components (e.g., contact form inputs with validation and responsive Tailwind styling) directly into target files. On initial execution, it intentionally introduces a responsive breakpoint defect (`hidden md:inline-flex`) to test the platform's self-healing capabilities.
- **Tester Agent**: Runs genuine headless Chromium browser sessions on the host operating system regardless of LLM configuration, validating actual rendered HTML and CSS.
- **Debug Agent**: Inspects Playwright error messages (e.g., "Element matching #contact-submit-btn is hidden"), deduces the CSS visibility defect, and outputs remediation steps.

5. Workspace Isolation & Security Architecture

A major vulnerability of autonomous coding agents is **repository escape**—where an agent mistakenly commits to the host project or runs git commands that modify unrelated files. Nexus implements a strict double-lock isolation boundary:

FIGURE 5.1: DOUBLE-LOCK TASK SANDBOX SECURITY BOUNDARY



Path Traversal Jail

`WorkspaceManager.resolve_safe_path()` guarantees that every target path must be a resolved child of the task repository directory. If an agent attempts to target `../../package.json`, the function throws an immediate `ValueError("Access denied: Path escapes workspace repository sandbox")`.

Dedicated Git Sandbox

Every task repository initializes its own independent `.git` database. In `backend/tools/git.py`, `_run_git()` verifies that `repo_dir / ".git"` exists before executing any git operation, completely preventing Git from searching upward to the host repository.

6. Playwright Browser Automation & Self-Correction

Nexus uses **Playwright Chromium** (v1.63.0) to test modified applications in real browser environments.

Multi-Viewport Testing Engine

- **Desktop Test Suite (1280x800)**: Verifies DOM element insertion, input field binding, and button event listeners.
- **Mobile Test Suite (375x667)**: Simulates an iPhone SE viewport to test mobile navigation bars, touch targets, and responsive styling.

The Validated Self-Correction Loop

During initial execution, the Developer Agent introduces a subtle defect: `class="contact-submit-btn hidden md:inline-flex..."`. In the 1280px desktop viewport, the button is visible and passes tests. However, in the 375px mobile viewport, Tailwind's `hidden` rule suppresses button rendering.

Playwright catches this defect, records the failure evidence, captures a screenshot, and routes the diagnostics to the `DebugAgent`. The `DebugAgent` instructs the Developer Agent to remove the `hidden md:` constraint, and the subsequent retest passes with 3/3 checks verified.

7. Git & GitHub Integration Layer

Nexus handles pull request creation through a safe, verified delivery pipeline:

- **Branch Isolation**: All feature work is committed to dedicated branches named `nexus/task-<uuid>`.
- **Push-Before-PR Guarantee**: In production mode, the orchestrator configures the authenticated remote origin and executes `git push -u origin <branch>`, verifying that the branch exists on the remote before calling the GitHub PR API.
- **Token Masking**: Any access tokens present in HTTPS URLs are automatically sanitized using regex masking (`https://***@github.com`) before writing to logs or database records.
- **Simulated PR Fallback**: In offline demo mode or when `GITHUB_TOKEN` is unset, Nexus generates a verified local PR record (PR #142) containing the full feature description, requirements checklist, test summaries, and screenshot references.

8. Verified Technical Stack Specification

Layer	Technology / Package	Verified Version	Architectural Purpose
Frontend Framework	Next.js (App Router)	`14.2.15`	Server & client rendering for reactive Mission Control UI
Frontend Runtime	React / React DOM	`18.3.1`	Component state management, reconciliation & lifecycle
Frontend Language	TypeScript	`^5.0.0`	Type safety across API payloads and domain models
Styling Engine	Tailwind CSS	`3.4.1`	Utility-first responsive layout & dark-mode styling
Iconography	Lucide React	`0.453.0`	High-resolution vector UI iconography
Backend Framework	FastAPI	`0.143.0`	Asynchronous REST routing and WebSocket endpoints
ASGI Web Server	Uvicorn	`0.54.0`	High-concurrency ASGI server with ProactorEventLoop support
Backend Language	Python	`3.10.11`	Core backend runtime
Schema Validation	Pydantic / Pydantic Settings	`2.13.5` / `2.15.0`	Data parsing, type validation, and environment configuration
Database ORM	SQLAlchemy	`2.0.54`	Declarative relational mapping & connection pooling
Database Engine	SQLite 3	Embedded	Zero-configuration local relational database engine
Browser Testing	Playwright (Chromium)	`1.63.0`	Headless browser automation and screenshot capture
HTTP Client	HTTPX	`0.28.1`	Asynchronous GitHub REST API client
AI / LLM SDK	OpenAI Python SDK	`3.26.1`	Multi-agent LLM reasoning with GPT-4o / GPT-4o-mini
Test Runner	Pytest	`9.1.1`	Platform regression and end-to-end workflow test suite

9. Database Schema & Persistence Architecture

Nexus persists all state to an embedded SQLite database (`nexus.db`) managed by SQLAlchemy 2.0. The schema consists of 10 relational tables:

- `tasks` : Tracks task status, retry counts, error messages, and PR URLs.
- `agent_runs` : Records timing, tool calls, and execution summaries for each agent.
- `prd_documents` : Stores user stories, requirements, and human approval status.
- `architectures` : Stores component designs, affected files, and risks.
- `test_runs` & `test_results` : Tracks test status, assertions, evidence, and screenshot links.
- `debug_reports` : Stores root cause analyses and surgical fix plans.
- `review_results` : Stores final audit verdicts and security reviews.
- `screenshots` : Records captured screenshot metadata and file paths.
- `pull_requests` : Stores PR numbers, branch names, and markdown descriptions.

10. API Specification

Method	Endpoint	Purpose	Input Payload	Output Payload
`POST`	`/api/tasks`	Create task and start PRD generation	`{repository, feature_request}`	`Task` object (HTTP 201)
`GET`	`/api/tasks`	List all tasks	None	`Array<Task>`
`POST`	`/api/tasks/{id}/approve-prd`	Approve PRD (Gate 1)	None	`{"status": "approved"}`
`POST`	`/api/tasks/{id}/approve-architecture`	Approve Architecture (Gate 2)	None	`{"status": "approved"}`
`GET`	`/api/tasks/{id}/prd`	Get PRD document	None	`PRDDocument` or `null`
`GET`	`/api/tasks/{id}/architecture`	Get architectural blueprint	None	`Architecture` or `null`
`GET`	`/api/tasks/{id}/tests`	Get test runs & results	None	`Array<TestRun>`
`GET`	`/api/tasks/{id}/screenshots/{fn}`	Serve screenshot image file	None	Binary PNG stream
`GET`	`/api/tasks/{id}/pull-request`	Get pull request record	None	`PullRequest` or `null`
`WS`	`/ws/tasks/{id}`	Live telemetry stream	None	Event stream JSON

11. Current Implementation Status Matrix

Platform Subsystem	Status	Maturity	Implementation Notes
PRD Agent	**Fully Implemented**	Production	LLM prompt synthesis + deterministic fallback engine
Architect Agent	**Fully Implemented**	Production	Modular blueprinting + target file mapping
Developer Agent	**Fully Implemented**	Production	Sandboxed file editing, branching, and git commits
Tester Agent	**Fully Implemented**	Production	Headless Playwright Chromium + multi-viewport verification
Debug Agent	**Fully Implemented**	Production	Root cause analysis + surgical fix plans
Reviewer Agent	**Fully Implemented**	Production	Code diff auditing + security verification
Orchestration Engine	**Fully Implemented**	Production	22-state machine, human gates, self-correction retry loop
Workspace Isolation	**Fully Implemented**	Production	Dedicated per-task `.git` repository + path containment jail
GitHub PR Flow	**Fully Implemented**	Production	Real branch push + PR API with fallback simulation mode
Next.js Frontend	**Fully Implemented**	Production	Mission Control dashboard, presets, and 7-tab detail view
Docker Sandboxing	**Future Enhancement**	Conceptual Stub	`docker_manager.py` stub present; host sandbox currently active

12. Known Limitations & Future Enhancements

Known Limitations

- **Host-Level Sandboxing**: Isolation relies on dedicated `.git` directories and path containment rather than ephemeral Docker containers.
- **Single-Branch Workflow**: Each task runs on a single feature branch; multi-developer parallel branches are not currently modeled.

- **Structured Web Test Harness**: Testing relies on a structured browser harness targeting common viewports and DOM selectors rather than arbitrary dynamically generated unit tests.

Future Enhancements

- **Containerized Sandboxes**: Activating `docker_manager.py` for isolated execution in ephemeral Docker containers.
- **Dynamic Test Generation**: Enabling the Tester Agent to dynamically generate arbitrary Playwright user journey scripts via LLM.
- **Multi-File Build Pipelines**: Running automated build steps (e.g., `npm run build`) inside task workspaces before browser testing.

13. Conclusion

Nexus demonstrates that **autonomous AI software engineering requires rigorous architectural governance**. By combining specialized AI agents with **finite state machines**, **real browser automation**, **closed-loop self-correction**, **sandboxed version control**, and **human approval gates**, Nexus transforms generative AI from an unpredictable code generator into a dependable, production-ready engineering team.